Tracing Planning in Python Policies

A static dependency-graph analysis (policy_graph: `planning-rubric-2`, `planning-scores-1`)

1 Motivation

Agent-generated Python policies can choose actions through very different computations: a direct rule, a search over hypothetical states, a lookup in a stored table, or a mix of these. The actions alone do not show which computation produced them. The planning-explanations write-up [1] addresses this by explaining a policy with *one fixed best-first search*, and by checking the explanation against the policy's execution and against the environment.

This note describes a complementary, static approach, implemented in `policy_graph`. It reads the policy's source and builds a *dependency graph* that records what every value is computed from. It then traces the returned action back through that graph, in the spirit of attribution graphs for language models [2], and reports planning only when a recognised planning computation lies on the action's dependence path. The policy is never imported or run. The analysis is deterministic, uses only the Python standard library, and labels the 35 policies of Section 8, together with the modules they import, in about 12 seconds.

For one policy, the analysis returns three things:

1. a *label*: planning; reactive; or needs review, when code that could not be analysed might change a reactive answer;
2. for every planning label, its *evidence*: the rule met, the code location, the dependence path to the action, and the assumptions under which this code counts as planning;
3. two *scores*, planning depth and planning work, when they can be counted exactly, and otherwise "unknown" with a reason.

The two approaches look at the same question from different sides. An explanation proposes an abstract search program and tests whether the policy's execution and the environment agree with it. It can measure how much search actually ran, and its claims rest on those tests. The dependency graph asks which computations the action depends on, across all branches of the code at once, and recognises planning by the structure of that dependence. It needs no simulator, test inputs or proposal search, but it reads the meaning of a computation from its form. Table 1 lists where each element of [1] appears here. The sections below follow the order of [1] where an element applies.

Table 1: Elements of the planning-explanations framework [1] and their counterparts in the static analysis.

Element of [1]	Counterpart here	Status
One fixed best-first search Q_θ (§2.1)	Its loop body, recognised in the graph (Definition 6)	static counterpart
State mapping α and checkpoints, Eqs. (1)–(2)	Frontier, selected node and discovered set, read from the dataflow (§3.2); not checked step by step	static counterpart
Output agreement, Eq. (3)	Evidence must lie in the backward slice of the returned action (Definition 2)	static counterpart
Interventions, Eq. (4) [6]	The dependence path from evidence to action is reported; nothing is intervened on	not applied
Abstractor ϕ and weak semantics, Eqs. (5)–(9)	Assumptions stated on each evidence item (§5)	not applied
Expansions N_H, M_H, Eq. (10)	Planning depth and work over hypothetical updates (§6.2); a different unit (§6.3)	different measure
Best explanation $S^*_{\mathcal{H}}$, Eq. (11)	One fixed analysis; no search over explanations	not applied
Admissibility restrictions (§4.2)	Admissibility of evidence (§6.4)	applied
Line-world worked example (§5)	The same policies, plus five more (§7)	applied

2 Program structure

In [1], program consistency maps the policy's execution states to states of the abstract search at chosen checkpoints, and checks that both evolve alike. The static counterpart is a single object built from the source: a dependency graph of the policy and the project modules it imports. Instead of aligning executions, the analysis looks in this graph for the dataflow that a planning computation must have.

2.1 The dependency graph

Definition 1 (Dependency graph)**.** For the analysed modules, the dependency graph is

$$G = (V, E, \kappa), \qquad E \subseteq V \times V, \qquad \kappa : E \to 2^K \setminus \{\emptyset\}, \tag{1}$$

with edge kinds $K = \{\mathsf{data}, \mathsf{control}, \mathsf{arg}, \mathsf{ret}, \mathsf{field_write}, \mathsf{base}, \mathsf{iter}\}$. An edge (u, v) means that v is computed from u, or that u decides whether v is computed. The value-flow kinds are $D = K \setminus \{\mathsf{control}\}$.

Each node has one kind and a source location:

- `def`: one node per assignment, so a variable assigned twice has two versions (`move@150`). Each return statement gives a node `<return>@`ℓ.
- `param` and `return`: a function's parameters, and its return value, which merges its return statements.
- `field` and `global`: a persistent attribute (`field:pkg.mod.Policy.plan`) or a module global.
- `call`: a call site (`call:f@`ℓ). A tuple unpacked from a call gets one element node per position (`call:f@`ℓ`[i]`), so the call stays on the path of each element.

- `op`: an addition or subtraction (`op+@`ℓ). It keeps its operands apart: $\mathrm{lhs}(o)$ and $\mathrm{rhs}(o)$ are the nodes whose values are copied into each operand.

The edge kinds are:

- data: u is read in the expression that defines v.
- control: u is used in a test that decides whether v runs: an enclosing `if` or `while`, a `for` iterable, or a test whose `return`, `break`, `continue` or `raise` guards later statements.
- arg and ret: an argument to its parameter, and a callee's return value to its call site.
- field_write: an assignment to a persistent attribute.
- base: an in-place change of a container (a mutating method, a `heapq` mutator, `c[k] = v`, `del c[k]`, a queue's `get()`) creates a new version of it, whose base is the old version (a weak update).
- iter: an iterable to the variable bound to each of its elements (loops, comprehensions, callback parameters).

A subset $Cp \subseteq E$ of *copy edges* carries a value unchanged: assignment, indexing, unpacking, returns, argument binding, conversions, `d.get(k)`, the result of `min`, `max` or `sorted`, and the element handed out by `pop`, `popleft` or `heapq.heappop`.

Code that cannot run is removed before the graph is used: statements after an unconditional `return` or `raise`, and branches whose test is a constant (a literal, or an attribute that can only hold one literal). Calls are resolved to project functions where possible. This includes function values stored in variables, parameters and attributes, and the callbacks of known higher-order calls such as `min(key=…)` or `scipy.optimize` objectives. Each call site records how it was resolved (project, builtin, library, method on an unknown object, dynamic, unresolved). The graph is context-insensitive: a helper called from two places has one body, whose values merge both calls.

2.2 The decision function and its action slice

For edge kinds $J \subseteq K$ and nodes $X \subseteq V$, let $\mathrm{Bwd}_J(X)$ be the nodes from which some node of X can be reached along edges with a kind in J, and $\mathrm{Fwd}_J(X)$ the nodes reachable from X. Without a subscript, all kinds are used.

Definition 2 (Action slice)**.** Let d be the decision function and r_d its return node. The action slice is

$$\mathcal{A} = \mathrm{Bwd}(\{r_d\}) \cup \{r_d\}. \tag{2}$$

The decision function is found by name (`get_action`, `act`, `select_action`, `choose_action`, `compute_action`, `policy`, then the generic `step`, `predict` and `__call__`, which also put the result under review), or it is set explicitly. $\mathcal{A}$ is a backward slice in the sense of program slicing [3, 4, 5]. It includes control dependence, so the action also depends on a planner when it depends only on whether a route was found. $\mathcal{A}$ plays the role of output agreement in [1, Eq. (3)]: a computation outside $\mathcal{A}$ cannot influence the returned action, so it cannot be evidence of planning.

2.3 State, candidates and hypothetical values

Definition 3 (State and candidates)**.**

i. The *state sources* Σ are the parameters of d except `self`, together with its *observation fields*: attributes of d's class that are assigned from values received by the class's non-constructor methods (`observe(self, obs): self.obs = obs`). Constructor parameters count as configuration. Both parts can be set explicitly.

ii. The *state-dependent* values are $S = \mathrm{Fwd}_D(\Sigma) \cup \Sigma$.

iii. The *state-like* values SL form the smallest set that contains Σ, contains v whenever $u \in SL$ and $(u, v) \in Cp$ is not an `iter` edge, and contains an operation o whenever $\mathrm{lhs}(o) \cap SL \neq \emptyset$, or o is an addition and $\mathrm{rhs}(o) \cap SL \neq \emptyset$. These are the state, a part or copy of it, and the state shifted by $+$ or $-$. An element drawn from a state collection by iteration is not state-like.

iv. The *candidate variables* Λ are the variables bound by iteration (loop, comprehension and callback variables) and optimiser decision variables.

v. A set of nodes N is a *pure candidate*, $\mathrm{pure}(N)$, if some non-field $n \in N$ depends on a candidate $\lambda \in \Lambda \cap \mathrm{Bwd}_D(\{n\})$ and on the state only through λ: n is not reached from $\Sigma \setminus \{\lambda\}$ by a value-flow walk that avoids λ.

Purity allows candidates that come from the state, as in `for a in obs["legal_actions"]`: a value computed from `a` alone is still a pure candidate.

Definition 4 (Hypothetical outcome)**.** An addition or subtraction o that does not build a list is a *hypothetical outcome* when one operand is state-like and the other is a pure candidate:

$$H = \big\{\, o : \ \big(\mathrm{lhs}(o) \cap SL \neq \emptyset \ \wedge \ \mathrm{pure}(\mathrm{rhs}(o))\big) \ \vee \ \big(\mathrm{rhs}(o) \cap SL \neq \emptyset \ \wedge \ \mathrm{pure}(\mathrm{lhs}(o))\big) \big\}. \qquad (3)$$

For example, `pos + a` with `a` drawn from `ACTIONS` is in H. So is `g - pos` for a candidate target `g`. Section 4.3 explains why that alone does not make a policy plan.

2.4 The claim: a planning rubric

Definition 5 (Planning rubric)**.** A policy *plans* when a reachable part of its implementation that influences the returned action

1. searches for or constructs a task plan, route, motion plan or control trajectory with a planning procedure (Rule 1); or
2. evaluates hypothetical consequences of candidate actions or plans and uses them to choose what to do (Rule 2). A horizon of one step is enough.

Otherwise the policy is *reactive*. Reactive covers rule-based controllers, fixed scripts, and rankings of actions that predict nothing, such as an argmax over logits, learned scores or heuristics of the present state.

Each rule is implemented by *recognisers* on G (Sections 3 and 4). A recogniser returns evidence items, and an item is admissible when its nodes meet $\mathcal{A}$. Let Ev be the admissible evidence and Rev the reasons for review: code in $\mathcal{A}$ that was not analysed and could change a reactive answer (§6.1). Then

$$\text{label}(P) = \begin{cases} \text{planning} & \text{if } Ev \neq \emptyset, \\ \text{needs review} & \text{if } Ev = \emptyset \text{ and } Rev \neq \emptyset, \\ \text{reactive} & \text{otherwise.} \end{cases} \tag{4}$$

Rule 2 is kept separate from Rule 1 because it needs no search procedure. A one-step lookahead builds no plan, yet it chooses by predicted consequences.

3 One best-first loop, recognised in the graph

The write-up [1, §2.1] fixes one generic best-first search and lets explanations vary its ingredients: the successor function, the goal, and the rule Best that selects a frontier node. The worklist recogniser takes the same stance from the other side. It fixes the loop body of that algorithm and recognises it in the dependency graph, however Best, the container and the removal are written. One definition covers breadth-first, depth-first, uniform-cost, A* and greedy best-first search, as well as searches over task sequences.

3.1 The worklist definition

Fix a `for` or `while` loop L in a function q, and a container C (a variable name) that is changed in the body of L. Let $\text{Own}(L)$ be the nodes of L's body outside nested loops.

- *Versions and sources.* V_C is the set of `def` nodes of C in q, and $\text{Src}_C = V_C \cup \{u : \mathsf{base} \in \kappa(u, v),\ v \in V_C\}$ adds the old versions that in-place changes start from. The changes in L are $\Delta_C = V_C \cap L$, and those in L's own body are $\Delta_C^\circ = \Delta_C \cap \text{Own}(L)$.
- *One pass.* $R_q(X; B)$ is the set of nodes reachable from X along value-flow edges within one pass through q. The walk may enter callees, but it does not leave q through its return or parameters, and it does not continue past B, fields or globals. R_q^{cp} is the same walk along copy edges only. The barrier stops a value from depending on itself by wrapping around the loop, or through a field into the next call.
- *Derived values and elements.* The values derived from C are $F_C = R_q(\text{Src}_C; \text{Src}_C) \setminus \text{Src}_C$, and its elements are $\text{El}_C = R_q^{cp}(\text{Src}_C; \text{Src}_C) \setminus \text{Src}_C$.
- *What a change receives.* For a change u, $\text{In}(u)$ is the set of value-flow predecessors of u outside Src_C, not counting the change's own call. $\text{keep}(u)$ holds when u has a value-flow predecessor in Src_C: it keeps the earlier contents.
- *Tables.* $\text{Tab}(L)$ is the set of names other than C that are written in L and tested in L, through `in` or `not in`, a subscript, or `.get` in an `if`, `while` or comprehension test. A table T is *kept*, $\text{kept}(T)$, when every version of T in L has a value-flow predecessor among the versions of T.

Definition 6 (Worklist search)**.** L is a worklist search over C when there is a `def` node x that is not a table, with successors $\text{Succ}_x = R_q(\{x\}; \text{Src}_C) \setminus \{x\}$, such that

$$\begin{aligned} &\text{(S) select:} && x \in \text{El}_C \cap \text{Own}(L), \\ &\text{(T) take out:} && \exists\, u \in \Delta_C^\circ :\ \text{keep}(u) \ \wedge\ \text{In}(u) \subseteq \{x\} \cup \big(F_C \setminus \text{Succ}_x\big), \\ &\text{(E) expand:} && \exists\, u \in \Delta_C :\ \text{In}(u) \cap \text{Succ}_x \neq \emptyset, \\ &\text{(R) remember:} && \exists\, T \in \text{Tab}(L) :\ \text{kept}(T). \end{aligned} \tag{5}$$

In words, in one iteration of the loop:

- **(S)** x is an element of C, taken from it unchanged: `C.pop(0)`, `heapq.heappop(C)`, `min(C, key=f)`, `C[i]`, `C.get()`. A value merely computed from C, such as `len(C)`, is not an element. The selection rule is free; this is where Best lives.
- **(T)** In the loop's own body, C changes in a way that keeps its other entries and brings in nothing except x itself, or values derived from C that do not depend on x, such as the index at which x was found: `pop`, `remove`, `discard`, `del C[i]`, `C = C[1:]`. A change that brings in anything else edits C, and `C = []` starts it afresh.
- **(E)** Somewhere in the loop, C receives values that depend on x, other than x itself: its successors.
- **(R)** The loop keeps a table of visited nodes, costs, parents or best results, which is written and tested in the loop and carried over between iterations. An array built afresh in every iteration is not memory of the search.

Names chosen by the policy's author play no role: `todo`, `parent` or `open` are never read. Each clause rejects a container loop that the other clauses accept. An iterative IK-style refinement fails (S). A growing history, and a list rebuilt in every iteration, fail (T). A work queue that adds no successors fails (E), and a task agenda without memory fails (R).

3.2 Correspondence with the fixed search

Table 2: The fixed best-first loop of [1, §2.1] and the clauses of Definition 6.

Step of the fixed search [1]	Clause	Remark
$v = \mathrm{Best}(F, G)$	(S)	any selection rule: FIFO, LIFO, priority, heuristic
Remove v from F	(T)	any removal: `pop`, `remove`, `discard`, `del`, slicing
For each (a, w) in $\mathrm{Succ}(v)$: append w to F	(E)	successors are values that depend on v
If w is new: add w and the edge (v, a, w) to G	(R)	a visited, cost, parent or best-result table
If $\gamma(v)$: return the action read from v	none	replaced by the slice condition (Definition 2)
Return the declared fallback	none	not required

The correspondence that the state mapping α declares in [1, §5.3] (the Python list as the frontier, the keys of the parent table as the discovered nodes) is what Definition 6 finds in the dataflow. The difference is in what is established. α is checked against executions at checkpoints and under interventions. Definition 6 is checked against the code, for all branches at once, but against no execution. A planning label therefore means that a search is reachable and influences the action, not that it ran on a given step.

There are two further differences. Definition 6 requires no goal test or fallback, because influence on the action is already required by $\mathcal{A}$. And it accepts reopening and cost relaxation, as in Dijkstra's algorithm or A* with re-insertion, which [1] excludes from its initial formulation.

3.3 The other Rule 1 recognisers

Table 3: Rule 1 recognisers. All evidence must lie in the action slice. The last column lists the library names that a recogniser relies on; none uses names chosen by the policy's author.

Recogniser	What is checked in the graph	Fixed names
planner call	a call listed in the planner contract table: `networkx` and `scipy.sparse.csgraph` shortest paths, `skimage` route search, OR-Tools routing, OMPL, `pybullet_planning`, Drake	about 50 library functions; extendable by configuration
worklist	Definition 6	none
relaxation	cost-table entries replaced by smaller candidates computed from earlier costs, while a predecessor table records where they came from (Bellman–Ford, dynamic-programming shortest paths, also vectorised)	none
layered expansion	each element of the current layer adds successors to the next layer, which becomes the current one	none
wavefront	a dilation applied again and again to the set it has reached (grid BFS, arrival times)	8 dilation functions, such as `binary_dilation`
tree search	recursion that branches over alternatives (in a loop or comprehension, or from two call sites) and compares the branches: a minimum, maximum or best-so-far over results, an ordering test, or a test on a recursive result (backtracking)	none
ordering search	a choice among the orderings enumerated by `itertools.permutations`	`itertools.permutations`
generate-and-test	candidates ranked by cost are tried in order, and the first that passes a project feasibility check that walks along the candidate is returned	ranking functions: `sorted`, `argsort`, `.sort`, `nsmallest`

A list of planner-like name tokens (`plan`, `search`, `solve`, `rollout`, ...) is used only to raise a review when an unresolved call carries such a name. It never produces evidence.

4 Lookahead: choosing by a predicted outcome

Rule 2 is recognised as a chain: candidate → predicted state → score → choice → action. Every link is defined on the graph. Only the choices and the declared models involve names.

4.1 Predicted outcomes

A predicted outcome has one of three forms: an arithmetic update $o \in H$ (Definition 4); a closed-loop rollout (Definition 7); or a call to a *declared transition model*, a function that the user lists in the configuration, such as a simulator step. M denotes the calls to declared models.

Definition 7 (Rollout)**.** An addition or subtraction o in a loop L of function q is a *rollout update* when

i. it updates a variable with its own value (`y += dy`, `y = y + dy`), so the next iteration's left operand is o's result, and the variable is not a list being built up;

ii. $\mathrm{lhs}(o) \cap SL \neq \emptyset$;

iii. the step depends on a candidate from outside L: $\left(\mathrm{Bwd}_D(\mathrm{rhs}(o)) \cap \Lambda\right) \setminus \Lambda_L \neq \emptyset$, where Λ_L are the candidate variables of L itself;

iv. another additive self-update in the same loop depends on o within q: an *accumulator*, such as `steps += 1` or `cost += c`.

H_{roll} is the set of rollout updates.

The accumulator separates a simulation from a solver. An iterative inverse-kinematics loop converges to a value and accumulates nothing.

4.2 Choices

A choice ch has *winners* $W(ch)$, the values chosen, and *scores* $Sc(ch)$, the values compared. Choices are recognised in five forms:

- a selection function (`numpy.argmin`, `argmax`, `argsort` and their `nan` variants, `heapq.nsmallest` and `nlargest`, ...): Sc is the compared array, plus the returns of a key callback;
- `min`, `max`, `sorted` or `.sort` comparing alternatives: Sc is the return of the key callback, or else the arguments;
- `xs.index(m)` where `m` is the minimum or maximum of `xs`: Sc is `xs`;
- a best-so-far loop, `if c < best: best = c; choice = a`: W is the values assigned in the branch, and Sc the dependences of its test;
- an optimiser (`scipy.optimize`): Sc is the objective's return, and its decision variable is a candidate.

About thirty library functions are listed in total.

4.3 Lookahead evidence

For a choice ch, let $\mathrm{Pred}(ch) = \mathrm{Bwd}(Sc(ch)) \cap H$ and $\mathrm{Mod}(ch) = \mathrm{Bwd}(Sc(ch)) \cap M$. Let $\mathrm{Roll}(ch)$ be the rollout updates whose accumulators reach $Sc(ch)$ by value flow, whose loops are driven by the candidates of ch, and that do not contain ch inside their own loop. A two-argument clamp such as `max(a, b)` is not a choice for a rollout.

Definition 8 (Lookahead)**.** A choice ch is lookahead evidence when $W(ch) \cap \mathcal{A} \neq \emptyset$ and

$$\mathrm{Mod}(ch) \cup \mathrm{Roll}(ch) \neq \emptyset \qquad \text{or} \qquad \mathrm{Pred}(ch) \neq \emptyset \ \wedge\ \mathrm{Enc}\big(W(ch)\big). \tag{6}$$

The *encoding condition* $\mathrm{Enc}(W)$ holds when the chosen value reaches the action without being combined with the state again. With $P = (\mathrm{Fwd}(W) \cap \mathcal{A}) \cup W$, and skipping parameters v (binding an argument computes nothing),

$$\begin{aligned} &\mathrm{Enc}(W) \iff u \notin S \ \text{ for all } (u, v) \in E_W, \\ &\quad E_W = \big\{(u, v) \in E \setminus Cp : \ v \in P \setminus W,\ u \notin P,\ \kappa(u, v) \cap \{\mathsf{data}, \mathsf{ret}, \mathsf{iter}\} \neq \emptyset\big\}. \end{aligned} \tag{7}$$

The encoding condition separates choosing an action by its predicted outcome from choosing a target by its distance and then steering to it:

```
return min(ACTIONS, key=lambda a: abs(4 - (pos + a)))     # planning: the winner is the action

t = min(GOALS, key=lambda g: abs(g - pos))
return int(np.sign(t - pos))                              # reactive: the winner is a target
```

Both choices score a value in H (`pos + a` and `g - pos`). In the first, the winner is the action. In the second, the winner `t` is combined with the state again (`t - pos`) before it becomes the action, so Enc fails. That policy is reported reactive, with a *target-choice* note. Steering to the nearest goal is what a reactive controller does, while choosing among actions by where they lead is one-step planning. When the scores come from a rollout or a declared model, no encoding check is applied, so a target chosen by simulating the approach to it does count.

A choice that reaches the action without any predicted outcome is noted as a *ranking*. If its scores come from an unanalysed call that receives both a candidate and the state, the result needs review, because that call may be a transition model.

5 Environment interpretation: assumptions instead of witnesses

In [1, §3], an abstractor ϕ gives the abstract graph a meaning in the environment, and each abstract action needs a witness trajectory under weak semantics. The static analysis has no simulator and checks no witness. Instead, every evidence item lists the assumptions under which the recognised computation is planning *about the world*, so that a reader can reject them (Table 4).

Table 4: Assumptions recorded with the evidence, and the part of [1] that each stands in for.

	Assumption	Used by	Stands in for, in [1]
A1	The state sources Σ are the policy's view of the environment state.	all of Rule 2	the root condition root $=$ $\phi(s_0)$, Eq. (9)
A2	State $\pm$ candidate is the state after applying the candidate.	arithmetic lookahead	an abstract transition $\hat{T}(x, a)$ with a witness, Eq. (7)
A3	A self-update with an accumulator simulates a controller.	rollouts	a multi-step witness trajectory
A4	A listed library call searches for and returns a plan.	planner calls	the meaning of the library's search
A5	A declared function predicts the next state.	transition models	$\hat{T}$, as declared by the user
A6	A frontier, relaxation or wavefront loop searches over states or locations.	graph search	the meaning of the search nodes

5.1 Shape is not meaning

These assumptions are read from the form of the code, and the same form can mean different things:

- `obs + w[a] * v` has the form of A2, but it may be a linear action score. For this reason, updates modulated by the state, such as `p + R(θ) @ a` or `x + v * cos(θ)`, are not recognised, and the transition function has to be declared.

- A perception flood fill with a visited table satisfies Definition 6, and iterated smoothing with a dilation looks like a wavefront.
- Inverse kinematics and collision checks at a candidate pose are not read as predictions, because their form cannot tell a kinematic model from a heuristic of the present state. Section 8.2 shows the one policy in the corpus that this costs.

What [1] would add here is a test of meaning. A witness check for each recognised prediction would replay the candidate in a simulator, from states where the prediction is computed, and compare the predicted value with the abstraction of the outcome. That would turn A2, A3 and A5 from assumptions into checked claims (Section 10).

6 Bringing the parts together

6.1 Label and analysis status

The label follows Eq. (4). Its *status* is reported separately. It is *supported* when planning evidence was found, or when nothing unresolved could change the reactive label. It is *needs review* when there is no planning evidence but code in $\mathcal{A}$ could change that. The reasons for review are: an unresolved import or callback; an opaque scorer (an unanalysed call that receives both a candidate and the state); a general solver whose role depends on its model (cvxpy, CP-SAT, LP); an unresolved call named like a planner; a dynamic call or an ambiguous method while planning code exists elsewhere; a decision function found only by a generic name; and a parse error.

When planning evidence exists, these reasons become notes, because unresolved code cannot remove supported evidence. A result that needs review is never reported as reactive. Patterns that were considered and rejected stay visible as notes: a ranking, a target choice, recursion along one branch, enumeration without evaluation, and planning code that does not influence the action.

6.2 Measuring planning: depth and work

The two scores answer two questions. *Depth* asks how far ahead the policy reasons, and *work* asks how much hypothetical exploration it performs. Their scope is the recognised, action-relevant hypothetical updates in one decision invocation.

Definition 9 (Planning depth and work)**.** Let U be the update sites behind the lookahead evidence: $\mathrm{Pred}(ch) \cup \mathrm{Roll}(ch)$ for the choices whose winners are evidence winners. The scorer executes d abstractly, without running any policy code. It tracks constants, the shapes of literal collections, and a hypothetical depth $\delta(v) \in \mathbb{N} \cup \{\bot\}$ for every value, with $\delta = 0$ for the inputs. An evaluation of a site $o \in U$ yields a value of depth $\delta(\text{state operand of } o) + 1$, and it is one unit of work. Every other operation takes the largest depth among its operands. If the sequence $\tau = (o_1, \ldots, o_W)$ of site evaluations is determined statically, then

$$\mathrm{work}(P) = |\tau| = W, \qquad \mathrm{depth}(P) = \max_{1 \le i \le W} \delta_i, \qquad \delta_i = 1 + \delta(\text{state operand of } o_i). \tag{8}$$

Both are 0 when $Ev = \emptyset$ and $Rev = \emptyset$.

Repeated evaluations count again, and a shared prefix computed once counts once. Converting the chosen candidate into an action, and arithmetic on scores, add nothing. Returning only the first action of a five-step rollout does not reduce its depth to one. Figure 1 shows two cases. The scores are *unknown*, with a reason, when:

- the label needs review, or unresolved code lies in the slice;
- the evidence includes a graph or tree search, a planner call, an ordering search or generate-and-test. No update-count contract exists for these, and how much a search explores depends on the map and goal at run time;
- planning happens outside the decision call, as with a plan computed in `reset()` and followed later;
- a declared transition model is used, since it has no single-step counting contract;
- τ is not static: a loop bound or candidate count depends on data, or a branch changes the count; or the code uses recursion, lazy generators, iteration over sets, writes to mutable fields or aliased collections, or decorated or variadic helpers; or a budget of 10,000 abstract steps is exceeded.

Unknown is never reported as zero, and no partial total is given.

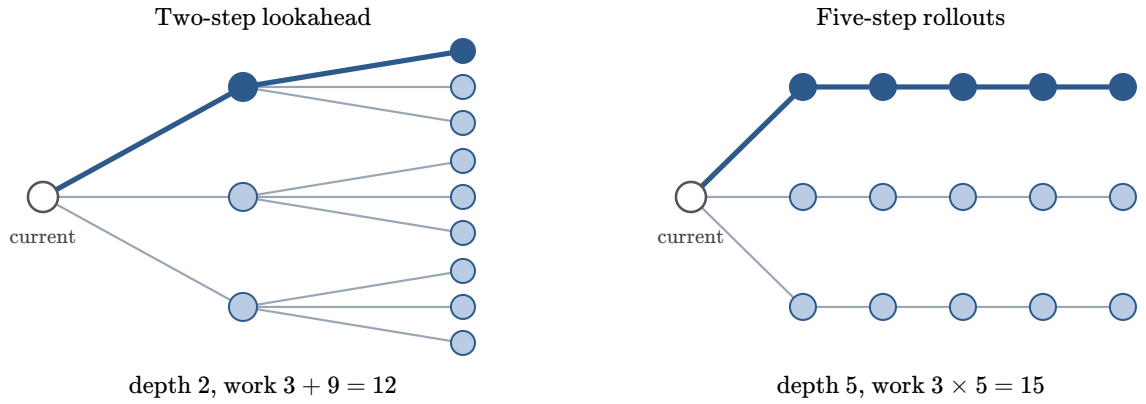


Figure 1: Planning depth and work for two line-world policies (Appendix A.4), as computed by the scorer. Each filled circle is one evaluation of a hypothetical update; the open circle is the current state, which is not counted. Depth is the longest chain (dark); work is the number of evaluations. The two-step lookahead is wide and shallow, and the rollouts are narrow and deep.

6.3 Relation to the expansion count

The write-up [1] counts N_H, the distinct environment nodes expanded in one invocation, and reports $M_H = \max(N_H - 1, 0)$. The units differ: work counts evaluations of hypothetical updates, that is, generated successors, while N_H counts expanded nodes. Suppose an explanation generates exactly one successor for each recognised update, and no label twice. Each expansion of a node v then generates $|\mathrm{Succ}(v)|$ successors, so

$$\begin{aligned} \text{work} &= \sum_{v \text{ expanded}} |\mathrm{Succ}(v)|, \\ \text{work} &= \sum_{i=1}^{k} b^{\,i} = b\,N_H, \quad \text{depth} = k \qquad \text{for a full tree of branching } b \text{ and depth } k. \end{aligned} \tag{9}$$

The two-step lookahead of Figure 1 has $N_H = 4$, $M_H = 3$ and work $12 = 3 \cdot 4$. The rollouts have $N_H = 13$, $M_H = 12$ and work 15. A one-step lookahead has depth 1 and work b, but $M_H = 0$. It plans under Rule 2, and it is the baseline under the coarse convention of [1], which measures "expansion beyond the immediate decision, not the number of actions considered at the root." These numbers are idealised: in the line world many two-step positions coincide, and the fixed search of [1] discovers each node only once.

6.4 Admissibility of evidence

The write-up [1, §4.2] restricts explanations so that they cannot manufacture their own planning. The static analysis places the corresponding restrictions on evidence:

- Evidence must lie in $\mathcal{A}$, which is computed after unreachable code is removed. Dead code, discarded results and unrelated functions produce at most a *not-action-relevant* note.
- Names never count. Identifiers chosen by the policy's author play no role. Library names enter only through the contract tables of Table 3 and Section 4.2, and each use is recorded as an assumption.
- Recursion, iteration, enumeration, numerical optimisation, copying, matching types and composition are not evidence on their own.
- Unknown is not zero, and needs review is not reactive.
- The configuration (state sources, planners, transition models, reactive scorers) is fixed across the policies being compared. Section 8 uses the default configuration for all 35 policies.
- Planning survives action conversion and caching. `2 * argmin - 1`, `ACTIONS[i]`, `path[1]`, `np.clip(waypoint - pos, …)` and plans cached in fields all keep a planning label. Rule 1 evidence needs no encoding check.

7 Worked example

7.1 Environment and policies

We use the line world of [1, §5.1]: positions $p \in \{0, \dots, 5\}$, actions -1, 0 and $+1$, transition $f(p, u) = p + u$ when the result stays in range, and the goal $p \geq 4$. The policies are the three of [1]: the reactive rule; P_{FIFO}; and P_{near}, which runs the same search with priority $2 - x$ (Appendix A.1–A.3). We add five line-world policies that span the range from reactive to planning (Appendix A.4): a nearest-target controller, a one-step lookahead, a two-step lookahead, five-step rollouts of each action, and a ranking by a stored cost table. Line numbers refer to the files in `policy_graph/cases/shared`, whose first two lines are a source note.

7.2 The one-step lookahead in the graph

Figure 2 shows the graph. Here $\Sigma = \{\texttt{pos}\}$. The callback's parameter `a` is bound by an iter edge from `ACTIONS`, so $\texttt{a} \in \Lambda$. The node `op+@3` has a state-like left operand and a pure-candidate right operand, so it is in H. The choice is `min` with a key: $W = \{\texttt{call:min@3}\}$, and Sc is the return of the callback, which depends on `op+@3`. The winner reaches r_d through copies only, so Enc holds. The report reads (module prefixes removed):

```
verdict: planning   status: supported
planning depth: 1   planning work: 3
[lookahead] look-ahead: candidates are scored by the predicted outcome ‘pos + a‘
            and chosen by min  (policy.py:3)
  prediction to scores:   op+@3 -> op-@3 -> call:abs@3 -> return:<lambda>  [data]
  chosen value to action: call:min@3 -> <return>@3 -> return:policy  [data]
  assumes: ‘pos + a‘ is read as the state after applying a candidate (additive transition)
           the chosen candidate becomes the action through a state-independent encoding
```

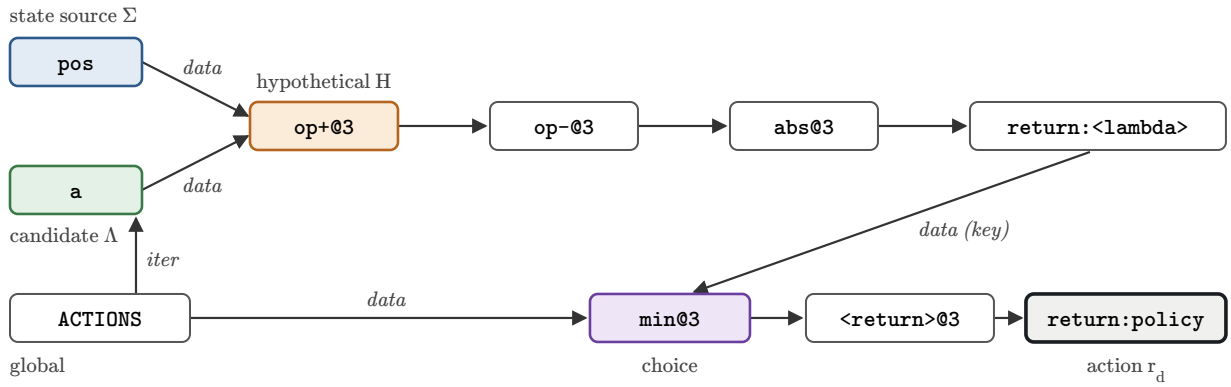


Figure 2: The dependency graph of the one-step lookahead `return min(ACTIONS, key=lambda a: abs(4 - (pos + a)))`, as built by the analysis. Every node lies in the action slice. All edges are data edges except the iter edge that binds the callback's parameter `a` to the elements of `ACTIONS`.

7.3 The two searches under Definition 6

The listing below is the search loop of P_{FIFO}, with the clauses of Definition 6 marked in comments. The comments are added here; the analysis never reads them.

```
    while todo:
        x = todo.pop(0)                          # (S) an element of todo; (T) taken out

        if x == 2:
            # Follow parents back to the first step.
            while parent[x] is not None and parent[x] != root:
                x = parent[x]
            return 0 if x == root else x - root

        for y in (x - 1, x + 1):
            if 0 <= y < 3 and y not in parent:  # (R) the table parent is tested ...
                parent[y] = x                    # (R) ... and written, keeping its entries
                todo.append(y)                   # (E) y depends on x: a successor goes in
```

Table 5: The clauses of Definition 6 in the two searching policies (Appendix A.2–A.3), and the step of the fixed search [1] that each one matches.

Clause	P_{FIFO}	P_{near}	Fixed search [1]
(S)	`x = todo.pop(0)` (l. 9): a copy of an element	`x = min(todo, key=priority)` (l. 8)	$v = \text{Best}(F, G)$
(T)	the same `pop`: a new version of `todo` that keeps the rest and receives nothing	`todo.remove(x)` (l. 9) receives only `x`	remove v from F
(E)	`todo.append(y)` (l. 20); `y` is `x - 1` or `x + 1`	`todo.append(y)` (l. 17)	append w to F
(R)	`parent[y] = x` (l. 19), tested by `y not in parent` (l. 18)	the same (l. 16, 15)	add w and its parent edge to G

Both policies are labelled planning with status supported, and with the evidence "graph search: an element `x` (line 9) is selected from frontier `todo`, taken out, and its successors are put in, with visited/cost table `parent`" (line 8 for P_{near}). The parts that α maps in [1, §5.3], `todo` to the frontier and the keys of `parent` to the discovered nodes, are exactly the container and the table that Definition 6 finds. In P_{near}, the priority is passed in as a parameter, so the callback in `min(todo, key=priority)` cannot be resolved. This adds a note but does not affect the label. Both scores are unknown, because no update-count contract exists for graph search.

7.4 The reactive policy

For `return 1 if pos < 4 else 0`, the slice $\mathcal{A}$ contains `pos`, the comparison and the return. There is no choice, no hypothetical operation and no loop, so the label is reactive with status supported, depth 0 and work 0. The write-up [1, §5.5] explains the same policy with a root and one terminal child, so $M_H = 0$.

7.5 Comparing the two measures

Table 6: The line-world policies. Label, evidence and scores are from the analysis. M_H is from [1]: the mean over start positions 0–5, with the raw N_H in parentheses; "–" marks policies that [1] does not treat.

Policy	Label	Evidence or note	Depth	Work	M_H [1]
Reactive rule (A.1)	reactive	none	0	0	0 (1)
Nearest target (A.4)	reactive	target-choice note	0	0	–
Cost table (A.4)	reactive	ranking note	0	0	–
One-step lookahead (A.4)	planning	lookahead: `pos + a`, chosen by `min`	1	3	–
Two-step lookahead (A.4)	planning	lookahead: `p1 + a2`, chosen by `min` and `.index`	2	12	–
Five-step rollouts (A.4)	planning	rollout: `p + a`, chosen by `.index`	5	15	–
P_{FIFO} (A.2)	planning	worklist: `todo`, `parent`	unknown	unknown	2/3 (4/3)
P_{near} (A.3)	planning	worklist: `min` + `remove`	unknown	unknown	1/3 (1)

Table 7: The two robot-navigation policies of [1, Appendix A.3–A.7]. Expansions are as reported there. Label, evidence and scores are from the analysis, with line numbers of the copies in `policy_graph/cases/shared`.

Policy	Expansions [1]	Label	Evidence	Depth, work
`E_astar_through_simulator`	4	planning	A* through the simulator, in `reset()` (approach.py:92)	unknown: planned outside the decision call
the same, `_simulate` declared	–	planning	also transition-model calls at approach.py:99 and 120	unknown: planned outside the decision call
`A_astar_fixed_lattice`	null (measure 25,093.7)	planning	grid A* in `reset()` (approach.py:89)	unknown: unresolved code in the slice

Where both measures give an answer, they agree. The reactive rule has no expansions and no planning, and every search labelled planning has a positive expansion count. They differ in what is counted and in what supports the count. `E_astar_through_simulator` shows the gap between static and dynamic. In `reset()`, the policy first rolls out its own reactive controller in a copy of the simulator, and runs the A* search only if the rollout is not short enough; `get_action` then follows the plan. On the development seeds, the A* branch did not run. The analysis's default evidence is that A*: it is reachable, and its plan influences the action. The rollout that did run steps the simulator through a project helper, and it is recognised only once that helper is declared as a transition model.

8 Results on 35 agent-written policies

8.1 Labels

The corpus is the final policy of each of 35 development runs by three coding agents (Claude Opus 5.5, GPT-6 Astra and GPT-5.6-sol) on four environments: Rovers, Obstruction2D, Obstruction3D and DynPushT2D. All 35 are analysed with the default configuration. There are two references, and neither is a blind gold label:

- The *hand label* of the task planner. *Search* plans an ordering of several tasks ahead, *Greedy* rescores candidates and picks the next task, and *Fixed* follows an order not driven by the scene. It describes task-level strategy only, while the rubric also counts route and motion planning.
- A *planning-strength tier*, from a manual reading of each policy's code, done separately from the analysis. *Strong* plans several tasks ahead (exactly the Search policies); *Medium* searches for a path or rolls out a controller for one task; *Weak* plans minimally, or only in recovery; *None* has no planning element.

Table 8: Hand label against the analysis.

Hand label	Policies	planning	reactive	review
Search	12	12	0	0
Greedy	14	8	6	0
Fixed	9	5	4	0
Total	35	25	10	0

Table 9: Planning-strength tier against the analysis.

Tier	Policies	planning	reactive
Strong	12	12	0
Medium	11	11	0
Weak	3	2	1
None	9	0	9

The thirteen Greedy or Fixed policies labelled planning are not disagreements. Eleven of them plan routes or motions for one task, with A*, Dijkstra, breadth-first or wavefront search, dynamic programming, a generate-and-test path planner, or controller rollouts. The other two are weak planners (Obstruction3D Claude 444 and Astra 424). Against the tiers, the analysis agrees on whether a policy plans for 34 of the 35 policies. Six of the nine None policies check clearance at a hypothetical pose, without choosing among predicted outcomes. Over the 25 planning labels, the evidence is a graph search in 19 policies (13 worklists, 3 layered expansions, 3 wavefronts, 1 relaxation), a tree search in 5, a planner call in 4, lookahead in 3, and generate-and-test and an ordering search in one each. Appendix B lists every policy.

8.2 The disagreement: Obstruction3D Claude 222

From our reading, this policy is a weak planner, but the analysis labels it reactive. Before moving the arm, it tries a few candidate base poses and IK seeds, checks each straight joint move along its length with forward kinematics, and keeps the cheapest valid one (condensed from `_jplan_to`):

```
for bb in cands:                        # candidate base poses
    for sd in (q, self.q_nom):          # IK seeds
        qg, err = self._ik_world(sd, goal, Rt, bb)
        if err > 1e-4: continue
        dq = qg - q
        c = max(np.max(np.abs(dq)), costb)
        if best is not None and c >= best[0]: continue
        if not self._path_ok(q, base, q + dq, bb, ...): continue
        best = (c, q + dq, bb)          # keep the cheapest
```

The analysis misses it for three reasons:

1. *The choice is not seen.* "Keep the cheapest that passes" is written with `continue` guards, while the best-so-far form expects `if c < best: best = ….`
2. *The hypothetical states are not recognised.* They come from inverse and forward kinematics, not from state ± candidate, and kinematic solves are deliberately not read as predictions (§5.1).
3. *Declaring the kinematics over-fires.* Declared as transition models, the IK or forward-kinematics helpers turn the label to planning, but 38 to 43 evidence items fire, most of them not planning.

The lesson is the same as for the worklist: define the choice and the prediction by what they do, not by how they are written.

8.3 Scores

All 10 reactive policies have depth 0 and work 0. All 25 planning policies have unknown scores: 17 plan by a search that has no update-count contract (graph, tree, ordering, or generate-and-test); 4 plan outside the decision call; 2 have unresolved code in the slice; and 2 hit other counting limits (a write to a mutable field, and an update outside the resolved call tree). Exact counts exist for lookahead and fixed rollouts, but the corpus policies plan by search. The current scores therefore cannot order the corpus by tier.

9 Validation

The test suite has 220 tests, which pass (one is skipped) in about 1.3 seconds:

- 34 checks from a review of the earlier version, each with the label the rubric requires; 40 independent cases covering every rule; and 17 rewrite-invariance pairs (a lambda against a named or method key, a direct against an identity helper, nested against split selection, plain against annotated assignment, six action encodings, an alias against a direct call, a callback field against a method, a renamed class);
- one grid search with its frontier written in 11 ways (`pop(0)`, `pop()`, `popleft`, `heappop`, `min` + `remove`, a set with `discard`, `sorted(…)[0]`, an index + `del`, a slice, `queue.PriorityQueue`, a field), all recognised as one search, together with the §5.1 policy of [1] with its priority as an argument; and five container loops that are not searches, each rejected by one clause of Definition 6;
- invariance to unrelated modules and to classes with the same name; eight needs-review cases and their resolution by configuration; a transition model that returns a tuple;
- properties of the graph checked on the graph itself (copies across calls, control dependence of mutations, container changes by `del` and `get`, popped values as elements, unpacked calls kept on the path), and the depth and work scores.

The corpus labels of Section 8 are kept as a regression reference. These are targeted checks of specific behaviour, not an estimate of accuracy. Still missing are blind gold labels, rewrite invariance measured on the corpus policies, and an ablation of each clause on the corpus.

10 Limitations and next steps

Limitations.

- *Static.* A planning label means that a planning path is reachable under the analysed configuration and influences the action, not that it runs on a given step (§7.5).
- *Meaning from shape* (§5.1). Predictions are recognised by their form; simulators and kinematic models must be declared.
- *Context-insensitive calls.* A helper's body merges all its call sites. In `E_astar_through_simulator`, deleting the call to `_search` still leaves `_search` cited, through the shared `_simulate` helper.
- *No type inference.* Methods called on fields of unknown type can stay unresolved. The fallback planners of `A_astar_fixed_lattice`, called through `self.fallback.get_action()`, are reported only as not action-relevant.

- *Known misses of Definition 6*, all labelled reactive with status supported: a queue walked with a moving head index (`x = q[i]; i += 1`), a list iterated while it grows, removal by set difference (`C = C - {x}` reads as an insertion), removal inside a helper (`x = take_first(C)`), and a tree search that keeps no table.
- *Scores* exist only for fixed lookahead and rollouts.

Next steps.

1. *Locate statically, count dynamically.* Use the evidence sites as probes, and count their executions in each decision at run time (for example with `sys.monitoring`). This gives work and depth for searches, planner calls and cached plans, whose order can then be validated against the tiers of Section 8. The amount of search is not its strength, so the horizon and the level (task or motion) should be reported alongside.
2. *Context-sensitive slicing* [5], to keep calls to a shared helper apart.
3. *Witness tests* for recognised predictions (§5), borrowing the weak semantics of [1].
4. *Blind gold labels*, to measure accuracy rather than agreement.
5. *The definition itself.* Is a choice by a stored cost estimate (`COST[pos, a]`) planning? And is a route planned once in `reset()` and followed later planning at every step, or only at the first?

References

[1] Planning Explanations for Python Policies. Write-up, October 2026.

[2] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, et al. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025.

[3] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[4] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.

[5] Susan Horwitz, Thomas Reps, and David Binkley. Interprocedural slicing using dependence graphs. *ACM Transactions on Programming Languages and Systems*, 12(1):26–60, 1990.

[6] Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 2021.

A Policy source listings

The line numbers match the analysed files. The first two lines of each file in `policy_graph/cases/shared` are a source note and are omitted here.

A.1 Reactive rule

`policy_graph/cases/shared/line_world_reactive/policy.py`

```
  3 def policy(pos):
  4     return 1 if pos < 4 else 0
```

A.2 P_{FIFO}

`policy_graph/cases/shared/line_world/policy.py`

```
  3 def policy(pos):
  4     root = pos // 2
  5     todo = [root]
  6     parent = {root: None}
  7 
  8     while todo:
  9         x = todo.pop(0)
 10 
 11         if x == 2:
 12             # Follow parents back to the first step.
 13             while parent[x] is not None and parent[x] != root:
 14                 x = parent[x]
 15             return 0 if x == root else x - root
 16 
 17         for y in (x - 1, x + 1):
 18             if 0 <= y < 3 and y not in parent:
 19                 parent[y] = x
 20                 todo.append(y)
 21 
 22     return 0
```

A.3 P_{near}

`policy_graph/cases/shared/line_world_near/policy.py`

```
def search(pos, priority):
    root = pos // 2
    todo = [root]
    parent = {root: None}
    while todo:
        x = min(todo, key=priority)
        todo.remove(x)
        if x == 2:
            while parent[x] is not None and parent[x] != root:
                x = parent[x]
            return 0 if x == root else x - root
        for y in (x - 1, x + 1):
            if 0 <= y < 3 and y not in parent:
                parent[y] = x
                todo.append(y)
    return 0


def policy(pos):
    return search(pos, lambda x: 2 - x)
```

A.4 Five more line-world policies

Each policy is a complete file.

Nearest target (reactive, target-choice note):

```
import numpy as np
GOALS = [4, 5]
def policy(pos):
    target = min(GOALS, key=lambda g: abs(g - pos))
    return int(np.sign(target - pos))
```

One-step lookahead (planning; depth 1, work 3):

```
ACTIONS = [-1, 0, 1]
def policy(pos):
    return min(ACTIONS, key=lambda a: abs(4 - (pos + a)))
```

Two-step lookahead (planning; depth 2, work 12):

```
ACTIONS = [-1, 0, 1]
def policy(pos):
    scores = []
    for a1 in ACTIONS:
        p1 = pos + a1
        scores.append(min([abs(4 - (p1 + a2)) for a2 in ACTIONS]))
    return ACTIONS[scores.index(min(scores))]
```

Five-step rollouts (planning; depth 5, work 15):

```
ACTIONS = [-1, 0, 1]
def policy(pos):
    costs = []
    for a in ACTIONS:
        p = pos
        for _ in range(5):
            p = p + a
        costs.append(abs(4 - p))
    return ACTIONS[costs.index(min(costs))]
```

Cost table (reactive, ranking note):

```
COST = {(p, a): abs(4 - (p + a)) for p in range(6) for a in (-1, 0, 1)}
def policy(pos):
    return min((-1, 0, 1), key=lambda a: COST[(pos, a)])
```

B The 35 policies

Table 10: Every corpus policy, with the hand label, the manual tier, and the analysis's label, evidence and reason for unknown scores. All labels have status supported. "Search" under scores means that no update-count contract exists for the evidence kind. Reactive policies have depth 0 and work 0.

Environment	Run	Hand	Tier	Label	Evidence	Scores unknown because
Rovers	Claude 24	Search	Strong	planning	layered BFS	unresolved code
	Claude 42	Search	Strong	planning	csgraph Dijkstra; worklist (stack)	unresolved code
	Claude 222	Search	Strong	planning	wavefront	search
	Claude 424	Search	Strong	planning	layered DP; wavefront; tree search	outside decision call
	Claude 444	Search	Strong	planning	wavefront; tree search (DFS)	search
	Astra 24	Greedy	Medium	planning	csgraph Dijkstra	outside decision call
	Astra 424	Search	Strong	planning	csgraph Dijkstra; worklist; tree search	outside decision call
	GPT-5.6 24	Fixed	Medium	planning	worklist (A*)	search
	GPT-5.6 424	Fixed	Medium	planning	worklist	search
Obstruction2D	Claude 24	Greedy	Medium	planning	relaxation (DP path)	search
	Claude 42	Greedy	Medium	planning	generate-and-test path	search
	Claude 222	Greedy	Medium	planning	rollouts (`y += dy`)	mutable field
	Claude 424	Search	Strong	planning	tree search; rollouts	search
	Claude 444	Search	Strong	planning	ordering search	search
	Astra 24	Greedy	None	reactive		
	Astra 424	Greedy	None	reactive		
	GPT-5.6 424	Fixed	None	reactive		
Obstruction3D	Claude 24	Greedy	None	reactive		
	Claude 42	Greedy	None	reactive		
	Claude 222	Greedy	Weak	reactive	missed (§8.2)	
	Claude 424	Greedy	None	reactive		
	Claude 444	Greedy	Weak	planning	layered BFS	search
	Astra 24	Fixed	None	reactive		
	Astra 424	Fixed	Weak	planning	lookahead (`hp[:2] + delta`)	update outside call tree
	GPT-5.6 24	Fixed	None	reactive		
	GPT-5.6 424	Fixed	None	reactive		
DynPushT2D	Claude 24	Search	Strong	planning	worklist; tree search	search
	Claude 42	Search	Strong	planning	worklist	search
	Claude 222	Search	Strong	planning	worklist	search
	Claude 424	Search	Strong	planning	csgraph Dijkstra; worklist	outside decision call
	Claude 444	Greedy	Medium	planning	worklist	search
	Astra 24	Greedy	Medium	planning	worklist	search
	Astra 424	Greedy	Medium	planning	worklist	search
	GPT-5.6 24	Fixed	Medium	planning	worklist	search
	GPT-5.6 424	Fixed	Medium	planning	worklist	search